Modern Cloud Computing

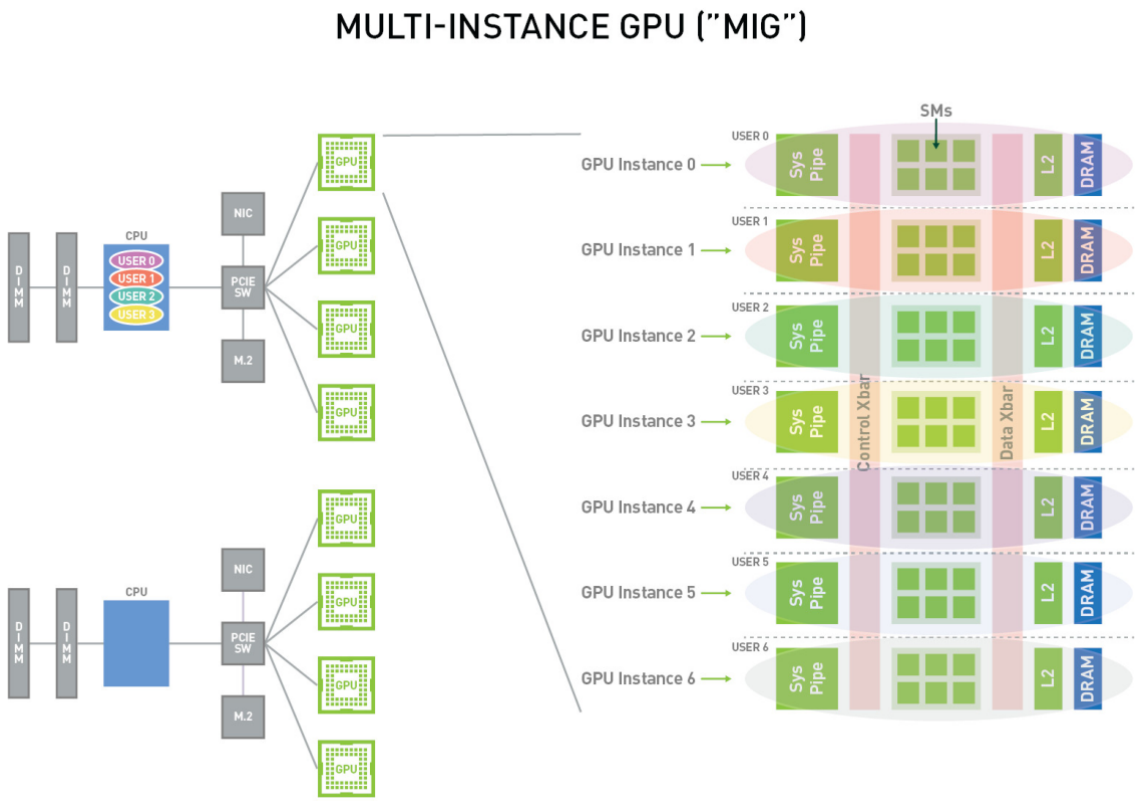
Lecture 16: Heterogeneous Computing in the Cloud I

Chongzhou Fang

Rochester Institute of Technology

Multi-Instance GPU (MIG)

- Ampere (A100, A30), Hopper (H100/H200), Blackwell (B200, GB200/GB300, RTX PRO)
- One GPU is securely split into **up to 7 GPU Instances**, each with its own SMs, L2 banks, memory controllers, DRAM busses and crossbar ports
- Predictable throughput and latency even if neighbors thrash caches or saturate DRAM; faults stay inside the instance
- Instances look like physical GPUs to CUDA, Docker, Kubernetes and hypervisors



(Source: NVIDIA MIG User Guide, Release 615, Fig. 1)

Sharing a GPU Among Tenants

Many inference, development and notebook jobs never saturate a powerful A100/H100 GPU. Cloud providers want to rent *fractions* of a GPU with whole-GPU guarantees.

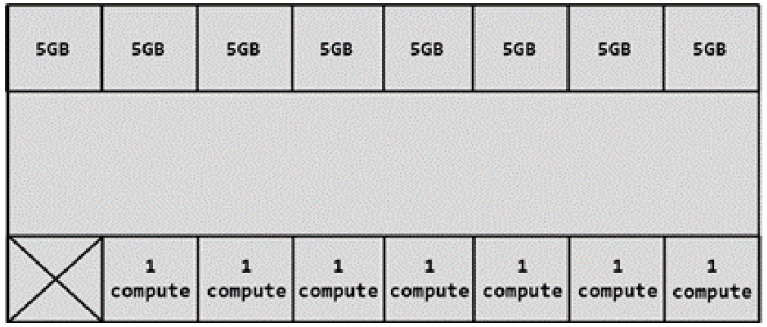
	Streams (CUDA programming model)	**MPS (CUDA Multi-Process service)**	**MIG**
Partition type	Single process	Logical	**Physical**
Max partitions	Unlimited	48	**7**
SM performance isolation	No	Yes (by percentage)	**Yes**
Memory protection	No	Yes	**Yes**
Memory bandwidth QoS	No	No	**Yes**
Error isolation	No	No	**Yes**
Cross-partition interop	Always	IPC	Limited IPC
Reconfigure	Dynamic	Process launch	When idle

(Source: NVIDIA Multi-Instance GPU User Guide, Release 615, Table 3)

GPU Terminology

Term	Meaning
Streaming Multiprocessor (SM)	Executes compute work: CUDA/tensor cores, registers, shared memory
GPC (Graphics Processing Cluster)	A cluster of SMs with shared front-end logic; MIG's SM slices follow GPC boundaries
GPU context	Like a CPU process: own address space and allocations; fault-isolated, individually scheduled
GPU engine	Compute/graphics, copy (DMA), NVDEC/NVENC/NVJPG/OFA; each engine scheduled independently
Memory system	L2 banks, memory controllers, DRAM busses, crossbar ports; assigned per instance by MIG
GPU memory slice	~1/8 of memory capacity *and* bandwidth, with its controllers and L2
GPU SM slice	~1/7 of the SMs available in MIG mode
GPU slice	One memory slice + one SM slice: the smallest unit of a partition
GPU Instance (GI)	Slices plus a share of engines; the unit of memory QoS and fault isolation
Compute Instance (CI)	A subset of a GI's SM slices; shares the GI's memory and engines

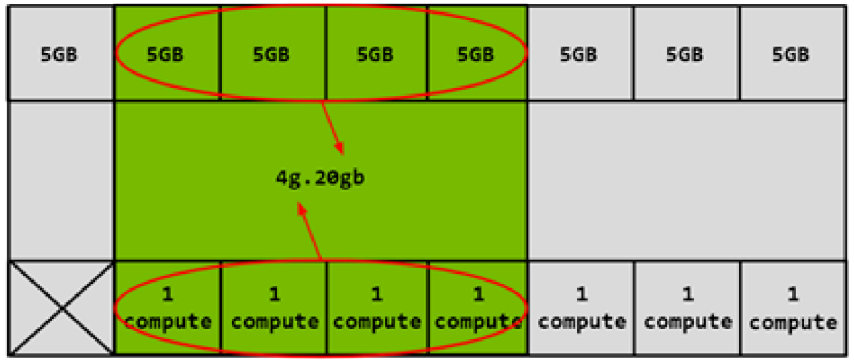
MIG Building Blocks: Slices



(Source: NVIDIA MIG User Guide, Release 615, Fig. 2)

- **Memory slice:** ~1/8 of capacity *and* bandwidth, with its memory controllers and L2
- **SM slice:** ~1/7 of the SMs (A100-40GB: 8 x 5 GB memory slices, 7 SM slices)
- **GPU slice** = one memory slice + one SM slice
- **GPU Instance (GI):** GPU slices plus a share of copy/NVDEC/NVJPG/OFA engines; the unit of memory QoS and fault isolation
- **Compute Instance (CI):** a subset of a GI's SM slices; shares its memory and engines

GPU Instance Profiles



4 memory slices + 4 compute slices = `4g.20gb` (Fig. 4)



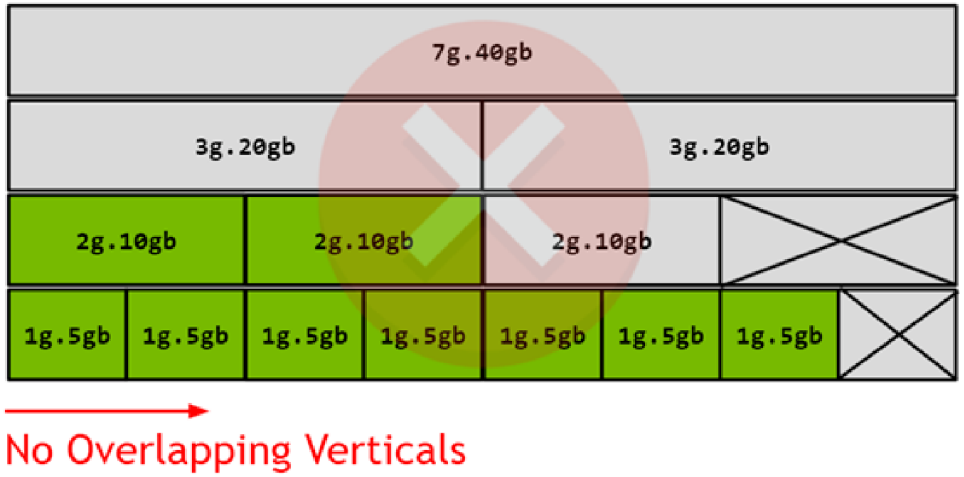
- 1 x 7g.40gb
  or
- 2 x 3g.20gb
  or
- 3 x 2g.10gb
  or
- 7 x 1g.5gb

Homogeneous geometries on A100-40GB (Fig. 7)

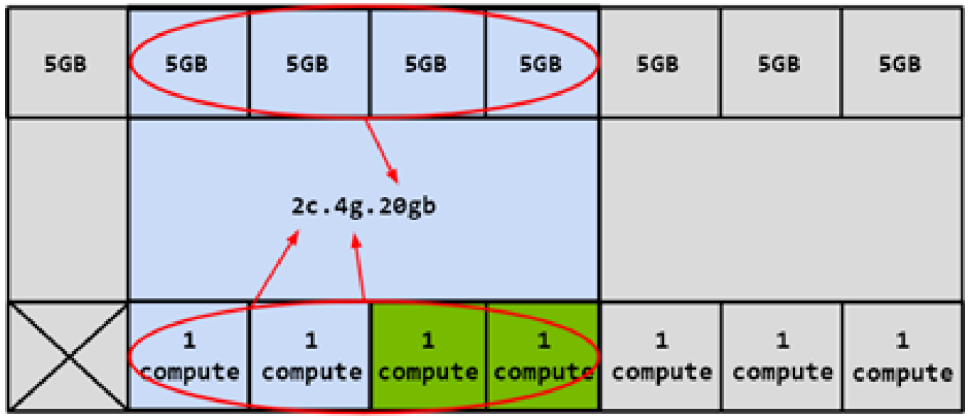
GIs are built from driver-defined **profiles**, not arbitrary slice counts (A100-40GB):

Profile	Memory	SMs	Copy engines	NVDEC / JPG / OFA	Max instances
`1g.5gb`	1/8	1/7	1	0 / 0 / 0	7
`2g.10gb`	2/8	2/7	2	1 / 0 / 0	3
`3g.20gb`	4/8	3/7	3	2 / 0 / 0	2
`4g.20gb`	4/8	4/7	4	2 / 0 / 0	1
`7g.40gb`	full	7/7	7	5 / 1 / 1	1

Placements, Compute Instances and Names



Mixed profiles are fine, but "no overlapping verticals" (Fig. 8)



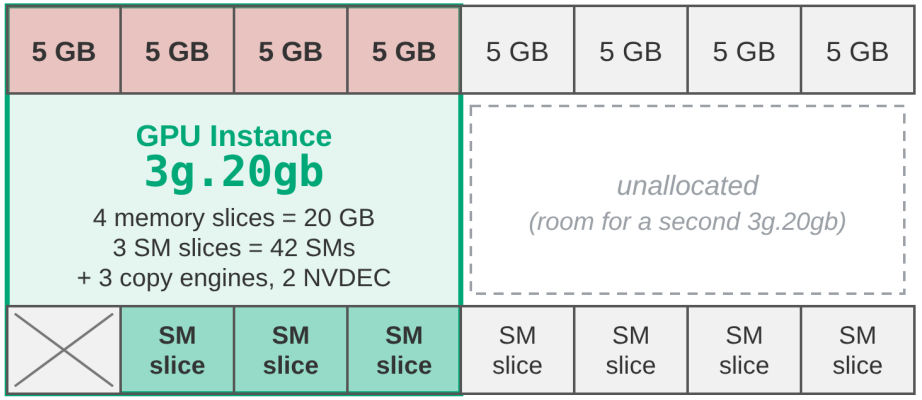
Two compute slices of a `4g.20gb` GI = `2c.4g.20gb` CI (Fig. 6)

- Geometries such as 3g + 2g + 1g + 1g are legal if nothing overlaps; a GI's physical position constrains what fits next to it, so fragmentation is possible
- Canonical name `<c>c.<g>g.<mem>gb`; `3g.20gb` alone means one CI using the whole GI
- Memory suffix follows the product: `1g.10gb` … `7g.80gb` on A100-80GB, up to `7g.251gb` on GB300

Reading a MIG Device Name: `1c.3g.20gb`

A100-40GB in MIG mode

8 memory slices x 7 SM slices; one 3g.20gb GPU Instance has been created



8 memory slices
(1/8 of capacity + bandwidth)

7 SM slices (GPCs)
(1/7 of the SMs; 14 each)

The 8th memory slice has no SM slice of its own, so 8 memory slices pair with 7 SM slices.

Inside the GI: Compute Instances

CIs own SMs; memory and engines stay shared

Split into three CIs (nvidia-smi mig -cci 0,0,0 -gi 1)

`1c.3g.20gb`	`1c.3g.20gb`	`1c.3g.20gb`
14 SMs	14 SMs	14 SMs

shared: 20 GB memory, L2, 3 copy engines, 2 NVDEC

Or an uneven split: 2c + 1c

`2c.3g.20gb`	`1c.3g.20gb`
28 SMs	14 SMs

shared: 20 GB memory, L2, 3 copy engines, 2 NVDEC

Default: one CI uses the whole GI, so the c part is omitted

`3g.20gb`
42 SMs (= 3c.3g.20gb)

shared: 20 GB memory, L2, 3 copy engines, 2 NVDEC

Reading the name

`1c.3g.20gb`

compute slices in this Compute Instance
1 x 14 SMs, private to the CI

GPU slices in the GPU Instance
3 SM slices (GPCs) = 42 SMs; sets engines too

total memory of the GPU Instance
4 slices x 5 GB (same GI is 3g.40gb on A100-80GB)

(Drawn from the definitions and A100-40GB profile data in the NVIDIA MIG User Guide, Release 615, Secs. 6.1-6.2 and Ch. 8)